

Sairam Hackathon 2026

Hackathon 2026 Challenge

Event:	iTech AI Innovation Hackathon 2026
Start Date:	1 August 2026
End Date:	7 August 2026
Team Size:	3 - 5 Members
Difficulty:	Intermediate to Advanced

Building Intelligent LLM Agents for Database Interaction & Visualization

1. Background & Context

Large Language Models (LLMs) have revolutionized how we interact with data. Modern AI agents can understand natural language queries, translate them into structured database operations, and present insights in human-readable formats. This hackathon challenges you to build an end-to-end conversational AI system that bridges the gap between non-technical users and complex databases.

The ability to query databases using natural language and instantly visualize results through charts and flowcharts represents a significant leap in business intelligence accessibility. Your solution should democratize data access, allowing anyone to extract insights without writing SQL or understanding database schemas.

2. Problem Statement

Build a ChatGPT-like conversational application where an LLM-powered agent can:

1. Understand natural language queries about data
2. Connect to and query SQL/NoSQL databases
3. Generate dynamic visualizations (charts, graphs, flowcharts)
4. Provide conversational explanations of the data insights

The core challenge is to design and implement custom tools (functions) for the LLM agent that enable database connectivity, query execution, and visualization generation.

3. Objectives

3.1 Primary Objectives

- **Chat Interface:** Build a responsive chat UI similar to ChatGPT for user

- **Agent Tools Development:** Create function-calling tools that the LLM can invoke
- **Database Integration:** Implement tools for schema discovery, query generation, and execution
- **Visualization Generation:** Create tools to generate charts (bar, line, pie, scatter) and flowcharts/diagrams

3.2 Secondary Objectives

- Handle multi-turn conversations with context retention
- Provide error handling and graceful fallbacks
- Support query explanation and SQL transparency

4. Functional Requirements

4.1 Chat Interface

- Real-time message display with streaming responses
- Message history persistence within a session
- Clear indication when the agent is processing/querying
- Embedded visualization rendering within chat

4.2 Required Agent Tools

Participants must implement the following tools (minimum):

Tool Name	Purpose	Expected Output
get_schema	Retrieve database schema (tables, columns, types)	JSON schema representation
execute_query	Execute SQL query against the database	Query results as JSON/tabular data
generate_chart	Create data visualizations (bar, line, pie, scatter)	Rendered chart image/component
generate_flowchart	Create flowcharts, ER diagrams, process flows	Rendered diagram (Mermaid/SVG)
explain_data	Generate insights and explanations from data	Natural language summary

4.3 Visualization Requirements

Data Charts (minimum 3 types):

- Bar Chart - for categorical comparisons

- Scatter Plot - for correlation analysis (bonus)

Flowcharts/Diagrams (minimum 2 types):

- Entity-Relationship (ER) Diagrams - visualize database relationships
- Process Flow Diagrams - illustrate data pipelines or workflows
- Decision Trees - for conditional logic visualization (bonus)

5. Technical Requirements

5.1 Technology Stack (Recommended)

Component	Options
LLM Provider	OpenAI GPT-4, Anthropic Claude, Google Gemini, or open-source models (Llama, Mistral)
Agent Framework	LangChain, LlamaIndex, CrewAI, AutoGen, or custom implementation
Backend	Python (FastAPI/Flask), Node.js (Express), or equivalent
Frontend	React, Vue.js, Streamlit, Gradio, or equivalent
Database	PostgreSQL, MySQL, SQLite (provided sample DB), or MongoDB
Charting Libraries	Plotly, Chart.js, Matplotlib, D3.js, Recharts
Diagram Libraries	Mermaid.js, Graphviz, Draw.io API, GoJS

5.2 Tool Implementation Guidelines

Each tool must follow these specifications:

- **Function Schema:** Define clear input parameters with types and descriptions
- **Return Format:** Structured JSON response that the LLM can interpret
- **Error Handling:** Graceful error messages that help the LLM recover
- **Documentation:** Clear docstrings explaining purpose and usage

6. Sample Use Cases

Your application should handle conversations like these:

Use Case 1: Sales Analysis

User: "Show me the top 5 products by revenue this quarter"

Agent: [Queries DB → Generates bar chart → Explains insights]

User: "Now show me the trend for these products over the last year"

User: "Draw me the ER diagram for this database"

Agent: [Fetches schema → Generates Mermaid ER diagram → Renders]

User: "Which tables are related to customers?"

Use Case 3: Process Visualization

User: "Create a flowchart showing how orders flow through our system"

Agent: [Analyzes tables → Infers process → Generates flowchart]

7. Evaluation Criteria

Criteria	Weight	Description
Functionality	30%	All required tools working, accurate query generation
Tool Design & Architecture	25%	Clean tool schemas, modular design, extensibility
Visualization Quality	20%	Chart clarity, appropriate visualization choices, aesthetics
User Experience	15%	Chat interface, responsiveness, error handling UX
Innovation & Creativity	10%	Novel features, creative solutions, bonus challenges

8. Submission Guidelines

8.1 Required Deliverables

- Source Code:** Complete codebase in a Git repository
- README.md:** Setup instructions, architecture overview, tool documentation
- Demo Video:** 3-5 minute walkthrough demonstrating key features
- Live Demo:** Working deployment or ability to run locally during judging

8.2 Code Requirements

- Clean, well-commented code following best practices
- Environment configuration via .env file (no hardcoded API keys)
- Docker support preferred for easy deployment

9. Resources Provided

- **Sample SQLite Database:** E-commerce dataset with orders, products, customers, and inventory tables
- **API Credits:** Limited credits for OpenAI/Anthropic APIs (or bring your own)
- **Starter Templates:** Basic project scaffolding for common tech stacks
- **Documentation:** Links to LangChain, function calling docs, Mermaid.js guides
- **Mentor Support:** Access to technical mentors during the hackathon

10. Bonus Challenges

Extra points for implementing:

- **Natural Language to SQL Explanation:** Show the generated SQL before execution
- **Multi-Database Support:** Connect to multiple databases simultaneously
- **Export Functionality:** Download charts as PNG/PDF, export data as CSV
- **Voice Input:** Speech-to-text query input
- **Query History & Favorites:** Save and rerun previous queries
- **Collaborative Features:** Share visualizations with team members
- **Custom Dashboard Builder:** Pin multiple visualizations to a dashboard

Team Name *

Team Member 1: *

	Student ID	Student Name	Department	Year	Student Contact Number	Student Mail ID
1						

Team Member 2: *

	Student ID	Student Name	Department	Year	Student Contact Number	Student Mail ID
2						